

Отчёт по этапу 1:

Разведочный анализ и предобработка данных

Проект: Построение и анализ ансамблевых моделей на основе решающих деревьев

15 июня 2026 г.

Датасет: Heart Disease Dataset (heart.csv)

Цель: Предсказание наличия сердечного заболевания (бинарная классификация)

Введение

В рамках первого этапа проекта был проведён разведочный анализ данных (EDA), выполнена предобработка и подготовка выборок для последующего построения ансамблевых моделей (Bagging, Random Forest, Gradient Boosting). В отчёте представлены результаты анализа пропусков, обработки категориальных признаков, разбиения данных, процедур валидации и выбора метрик качества.

1 Проблема пропущенных данных

1.1 Анализ пропусков

В ходе первичного анализа данных была проведена проверка наличия пропущенных значений во всех признаках с использованием метода `df.isnull().sum()`.

Таблица 1: Проверка пропусков по признакам

Признак	Количество пропусков	% от выборки
age	0	0%
sex	0	0%
cp	0	0%
trestbps	0	0%
chol	0	0%
fbs	0	0%
restecg	0	0%
thalach	0	0%
exang	0	0%
oldpeak	0	0%
slope	0	0%
ca	0	0%
thal	0	0%
target	0	0%

1.2 Вывод

Пропуски в датасете **отсутствуют**. Дополнительные процедуры заполнения (медианой, модой или с использованием моделей) не потребовались, что упрощает предобработку.

2 Обработка категориальных данных

2.1 Анализ типов признаков

Все признаки в датасете представлены в числовом формате. Однако по смыслу некоторые из них являются категориальными.

Таблица 2: Типы признаков

Признак	Тип данных	Уникальных значений
sex	int64	2 (бинарный)
cp	int64	4 (номинальный)
fbs	int64	2 (бинарный)
restecg	int64	3 (номинальный)
exang	int64	2 (бинарный)
slope	int64	3 (порядковый)
ca	int64	5 (порядковый)
thal	int64	4 (номинальный)
age, trestbps, chol, thalach, oldpeak	int64/float64	непрерывные

2.2 Выбранный метод кодирования

Для решающих деревьев и ансамблей (Random Forest, Gradient Boosting) категориальные признаки **не требуют** преобразования в One-Hot Encoding, так как деревья работают с пороговыми значениями и корректно обрабатывают числовые метки классов.

Принятое решение: Оставить все признаки в числовом виде без дополнительного кодирования.

3 Разбиение данных на тренировочную, валидационную и тестовую выборки

3.1 Методология

Для корректной оценки качества моделей и предотвращения утечки данных выборка была разделена на три части:

- **Обучающая (train)** — 60% данных, используется для обучения модели
- **Валидационная (val)** — 20% данных, используется для настройки гиперпараметров
- **Тестовая (test)** — 20% данных, используется для финальной оценки

3.2 Стратификация

Целевая переменная имеет распределение: класс 0 — 48,7%, класс 1 — 51,3% (см. Рисунок 4). Для сохранения пропорций классов во всех выборках использовалось **стратифицированное разделение** (параметр `stratify=y`).

Таблица 3: Размеры выборок

Выборка	Доля от данных	Доля класса 0	Доля класса 1
Train	60%	48,7%	51,3%
Validation	20%	48,7%	51,3%
Test	20%	48,7%	51,3%

3.3 Вывод

Разделение выполнено корректно, пропорции классов сохранены. Это позволит объективно сравнивать модели и избежать переобучения.

4 Процедуры валидации и кросс-валидации

4.1 Валидация на отложенной выборке

Основной подход к валидации в проекте:

1. Модели обучаются на **train**
2. Гиперпараметры настраиваются по **validation**
3. Финальная оценка проводится на **test**

4.2 Кросс-валидация

Для более стабильной оценки качества при подборе гиперпараметров будет использоваться **Stratified K-Fold Cross-Validation**:

- Количество фолдов: $k = 5$
- Параметр `shuffle = True` для случайного перемешивания
- Стратификация для сохранения пропорций классов в каждом фолде

Формула для K-Fold кросс-валидации:

$$CV_{score} = \frac{1}{k} \sum_{i=1}^k \text{Score}(\text{model}, \text{train}_i, \text{val}_i)$$

4.3 Зачем нужна кросс-валидация

- **Устойчивость:** усреднение по 5 разбиениям снижает влияние случайности
- **Эффективное использование данных:** каждый объект побывает и в обучении, и в валидации
- **Защита от переобучения:** раннее обнаружение проблемы на валидации

5 Метрики качества для задач классификации

5.1 Описание метрик

Для оценки качества бинарной классификации будут использоваться следующие метрики:

Таблица 4: Метрики качества

Метрика	Формула	Когда важна
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	При сбалансированных классах
Precision	$\frac{TP}{TP+FP}$	Минимизация ложных срабатываний
Recall	$\frac{TP}{TP+FN}$	Минимизация пропусков (особенно важно в медицине)
F1-score	$2 \cdot \frac{P \cdot R}{P+R}$	Гармоническое среднее (основной показатель)
ROC-AUC	Площадь под ROC-кривой	Разделяющая способность
PR-AUC	Площадь под Precision-Recall	При сильном дисбалансе

5.2 Матрица ошибок (Confusion Matrix)

	Предсказано	
	0	1
Реально 0	TN	FP
Реально 1	FN	TP

Где:

- **TP (True Positive)** — правильно предсказана болезнь
- **TN (True Negative)** — правильно предсказано здоров
- **FP (False Positive)** — ложная тревога
- **FN (False Negative)** — пропущенная болезнь (наиболее опасная ошибка в медицинском контексте)

5.3 Выбор приоритетной метрики

Учитывая медицинский контекст (диагностика сердечных заболеваний), наиболее важной метрикой является **Recall** (чувствительность), так как пропустить болезнь опаснее, чем ложная тревога.

Рекомендуемый порог качества для проекта:

- $F1\text{-score} \geq 0.85$
- $ROC\text{-AUC} \geq 0.90$

6 Результаты EDA

6.1 Рисунок 1 — Распределения признаков

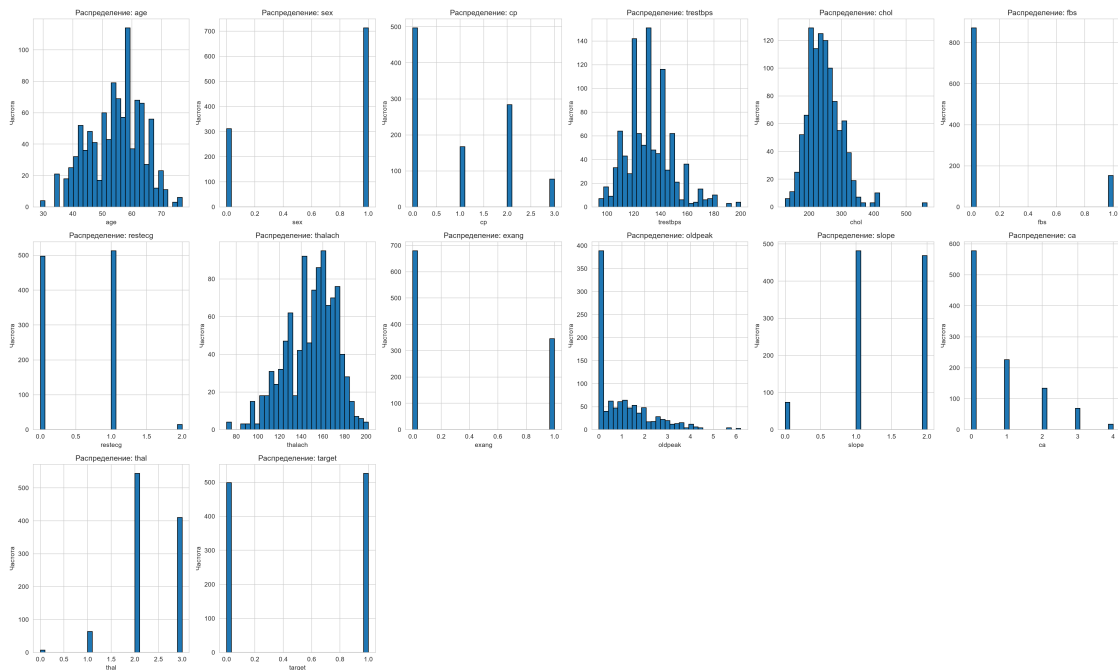


Рис. 1: Гистограммы распределений всех признаков

Анализ:

- age — распределение близко к нормальному, пик в районе 50–60 лет
- trestbps (давление) — асимметрия вправо, несколько выбросов выше 160
- chol (холестерин) — распределение с длинным правым хвостом
- thalach (макс. ЧСС) — почти нормальное распределение
- oldpeak — сильно смещено влево, много нулевых значений

6.2 Рисунок 2 — Boxplots

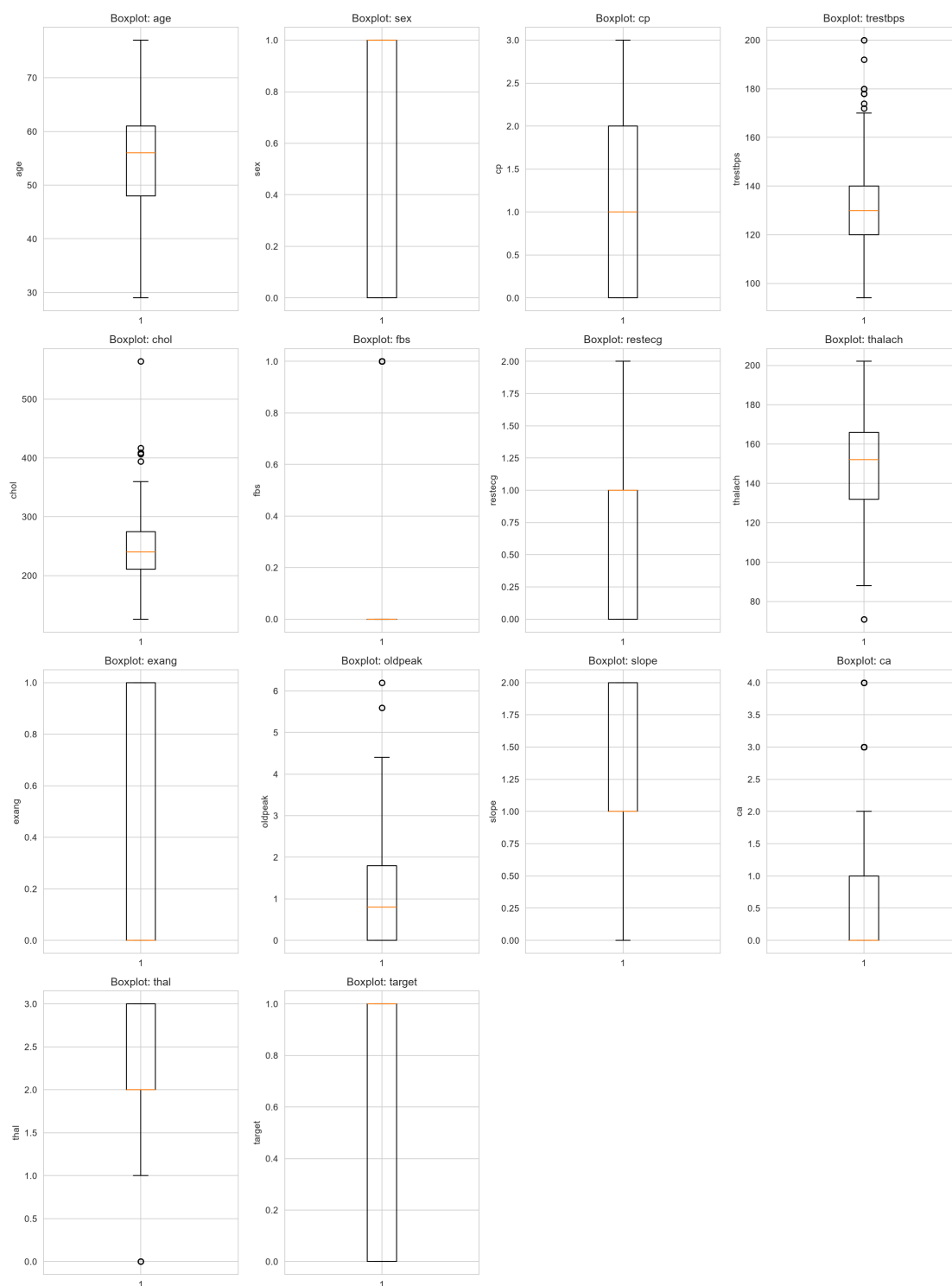


Рис. 2: Boxplot'ы для выявления выбросов

Выявленные выбросы:

- trestbps — несколько значений выше 180
- chol — значения выше 400
- oldpeak — значения выше 4

- **ca** — значения 3 и 4 (редкие)

Важно: Для решающих деревьев выбросы не критичны (алгоритм основан на порогах, а не на расстояниях).

6.3 Рисунок 3 — Матрица корреляций

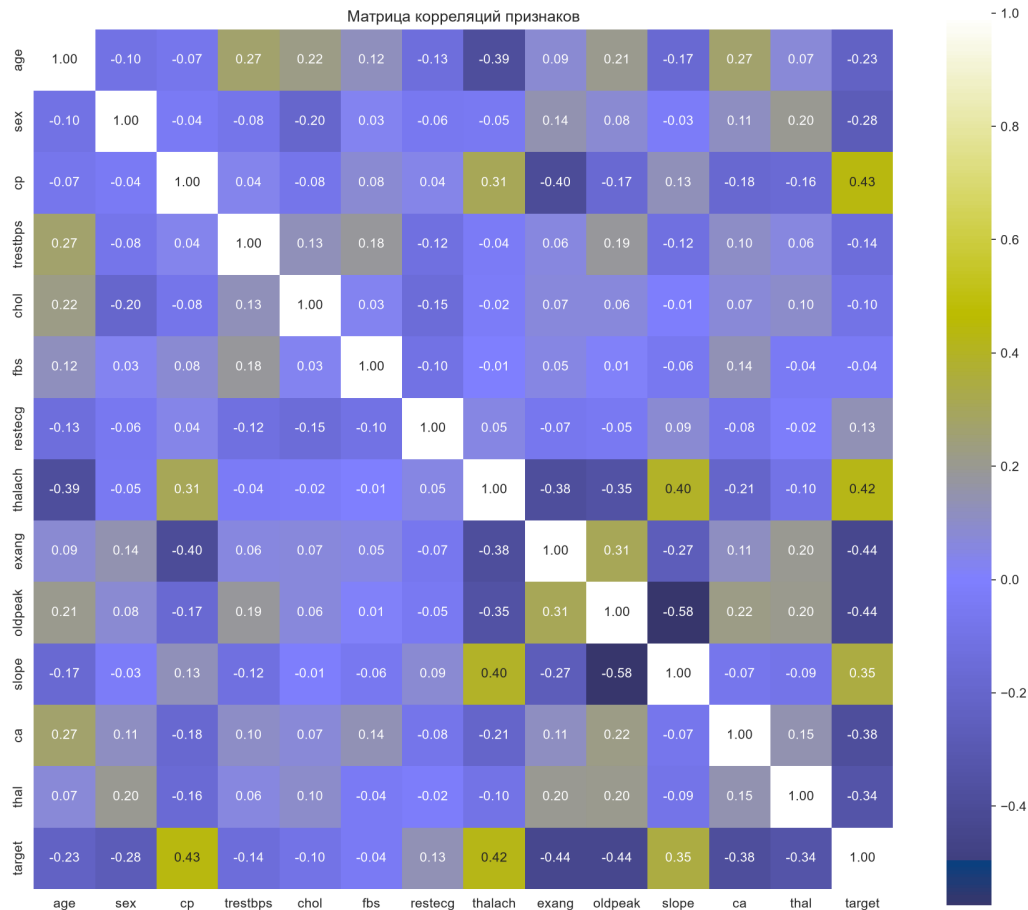


Рис. 3: Матрица корреляций признаков

Корреляции с целевой переменной (target):

- **cp** — +0.43 (умеренная положительная)
- **thalach** — +0.42 (умеренная положительная)
- **exang** — -0.44 (умеренная отрицательная)
- **oldpeak** — -0.44 (умеренная отрицательная)
- **ca** — -0.44 (умеренная отрицательная)

Мультиколлинеарность: Сильной корреляции ($|r| > 0.7$) не обнаружено. Все признаки можно оставить в модели.

6.4 Рисунок 4 — Распределение целевой переменной

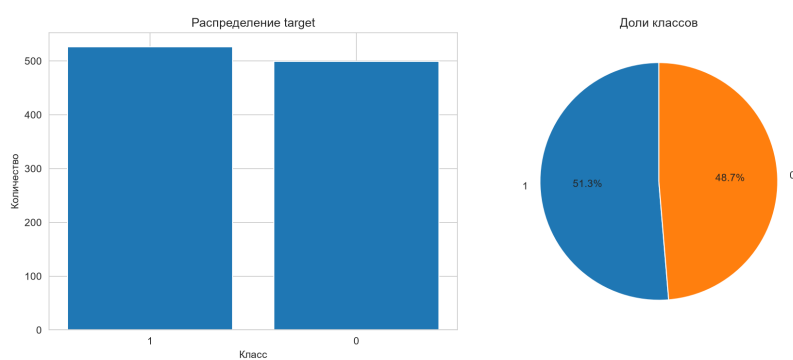


Рис. 4: Распределение целевой переменной target

- Класс 0 (нет болезни): 48,7%
- Класс 1 (есть болезнь): 51,3%

Вывод: Классы почти сбалансированы. Дисбаланс не критичный, можно использовать ассигасу, но предпочтительнее F1-score.

6.5 Рисунок 5 — Признаки по классам

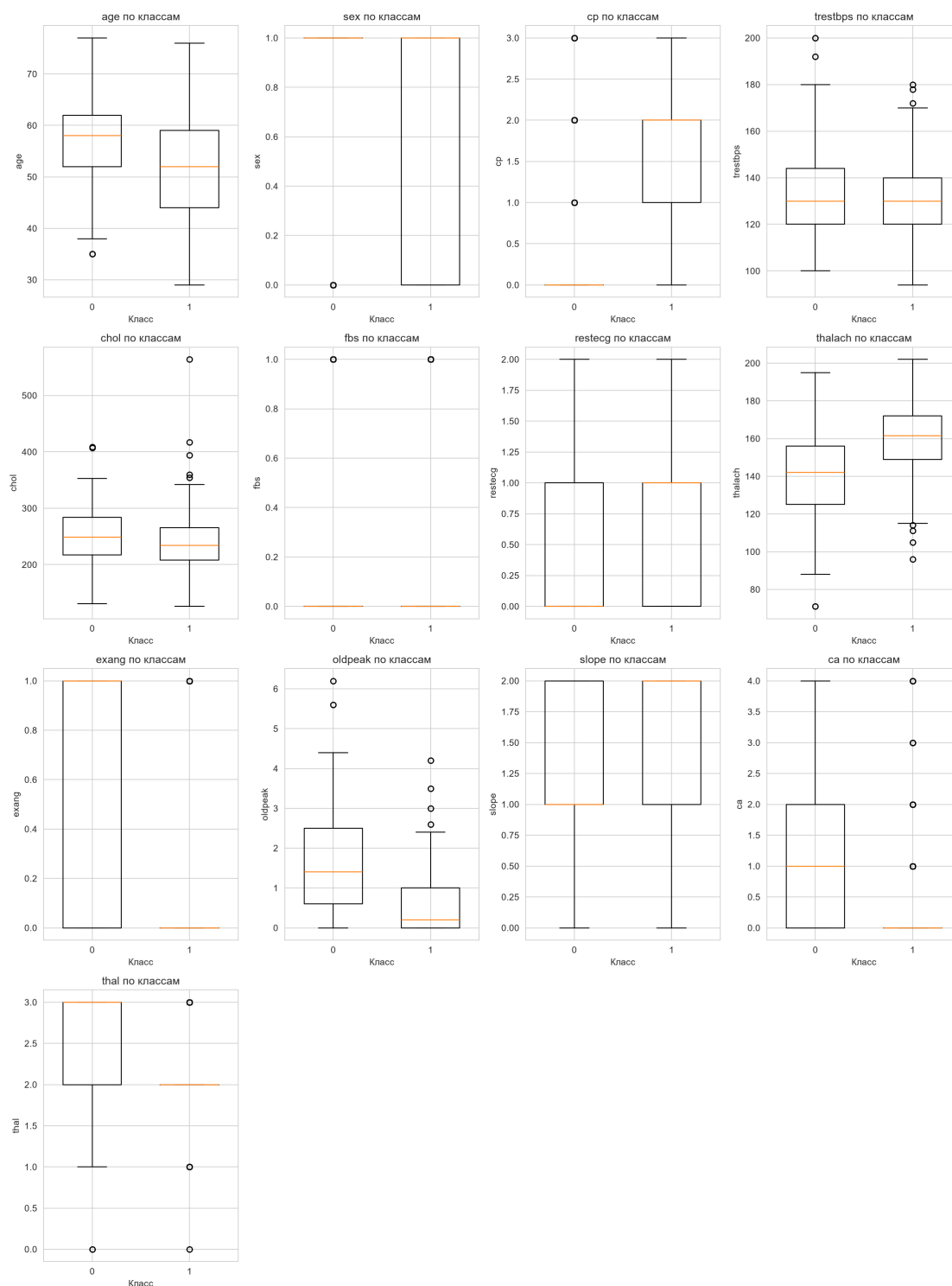


Рис. 5: Распределение признаков в разрезе классов target

Наглядно видно, что по большинству признаков распределения для больных и здоровых различаются:

- **thalach** — у больных ЧСС ниже
- **oldpeak** — у больных выше
- **exang** — чаще встречается у больных

Заключение по этапу 1

Критерий	Статус	Комментарий
Пропуски		Отсутствуют
Обработка категорий	Не требуется для деревьев	
Разделение train/val/test	60/20/20 со стратификацией	
Кросс-валидация		Будет применена на этапах 2–4
Метрики	Выбраны: F1, ROC-AUC, Recall	

Итоговая таблица выполнения этапа 1

Данные готовы для перехода к этапу 2 — обучению базовых моделей (решающих деревьев).

Приложение: Ключевые параметры для следующих недель

```
# Параметры разделения
TRAIN_SIZE = 0.6
VAL_SIZE = 0.2
TEST_SIZE = 0.2
RANDOM_STATE = 42
STRATIFY = True

# Кросс-валидация
CV_FOLDS = 5
CV_STRATIFIED = True
CV_SHUFFLE = True

# Основная метрика
PRIMARY_METRIC = 'f1_macro'
```